

aios-agent v2.2 — Resumen Ejecutivo

Propósito

aios-agent es un agente ligero de SRE que utiliza **function calling nativo** sobre un modelo local **Qwen** (Qwen3-8B por defecto, Qwen2.5-7B-Instruct alternativa) servido por llama.cpp. Responde en inglés/español/chino, ejecuta comandos Linux, lee archivos de configuración y logs, y escribe archivos bajo ciertas rutas permitidas.

Configuración inicial (Setup Wizard)

La primera ejecución ejecuta `setup.py`, un instalador en modo texto totalmente en inglés que permite elegir:

1. **LOCAL** — Usa el modelo Qwen incluido en la ISO. Mínimo **8 GB RAM**, 12 GB recomendados.
2. **CLOUD** — Seleccionar proveedor (DeepSeek, OpenAI, Anthropic, Google Gemini, Moonshot/Kimi, Ollama Cloud, OpenRouter) e introducir API key (entrada visible para pegar con comodidad).
3. **HÍBRIDO** — El modelo local actúa como orquestador; consultas complejas se delegan al cloud mediante la tool `cloud_reasoning`.

El wizard detecta automáticamente la RAM total desde `/proc/meminfo` y escala el contexto local:

- 8 GB → 8K tokens
- 12–16 GB → 32K tokens
- 16 GB → 64K tokens

Muestra en pantalla: `Detected: 14 CPU threads, 62 GB RAM -> 64K context`. También ajusta los threads de CPU al 87.5% de los cores disponibles (p. ej. 14/16 cores) y presenta el uso como `N/16 cores` en lugar de un porcentaje.

El wizard guarda el `context_limit` específico de cada proveedor en `~/.aios/config.yaml`. Las API keys se guardan aparte en `~/.aios/.env`, que está en `.gitignore` y nunca se commitea. Si el usuario deja la API key vacía en modo cloud/híbrido, el agente cae de forma elegida a modo local.

Selección de modelo local

Durante el setup se puede elegir:

Modelo	Por defecto	Características
Qwen3-8B Q4_K_M	Sí	Más fiable para function calling; se descarga automáticamente desde HuggingFace si falta.
Qwen2.5-7B-Instruct Q4_K_M	Alternativa	Más rápido; útil cuando la velocidad prima sobre la fiabilidad máxima de tool calls.

ISO Integration

El repositorio incluye todo lo necesario para empaquetar el agente como imagen ISO bootable que arranca automáticamente.

Archivo	Función
<code>systemd/aios-llama.service</code>	Lanza el servidor llama-server llamando a <code>scripts/launch_llama.py</code> .
<code>systemd/aios-agent.service</code>	Espera a que <code>http://127.0.0.1:8083/health</code> responda y luego ejecuta <code>chat.py</code> .
<code>scripts/launch_llama.py</code>	Lee <code>~/aios/config.yaml</code> , arma el comando de llama-server y hace <code>exec</code> con <code>--jinja</code> .
<code>scripts/firstboot.sh</code>	Wizard de primer arranque; ejecuta <code>setup.py</code> , habilita e inicia los servicios si el modelo ya está descargado.

Comportamiento:

- `aios-llama.service` se ordena tras `network-online.target` y reinicia ante fallos.
- `aios-agent.service` depende de `aios-llama.service`; usa un bucle `curl` hasta que el servidor local responde.
- Ambos servicios corren como usuario `aios` con el código en `/usr/local/bin/aios-agent`.
- La interfaz, prompts y mensajes de salida están en inglés para un comportamiento consistente sin depender de la locale del host.

Modelo definitivo

- **Qwen3-8B Q4_K_M** por defecto, o **Qwen2.5-7B-Instruct-Q4_K_M** alternativa.

- Servidor llama.cpp con `--jinja` y `-t 14` sobre 16 cores.
- Qwen3-8B / Qwen2.5-7B-Instruct emiten `tool_calls` válidos de forma consistente y completan planes multi-paso.

Cuantización del modelo

Cuantización	Velocidad (tok/s)	RAM/VRAM usada	Calidad de respuesta	Caso de uso
Q4_K_M	57 prompt / 20 gen	4.7 GB	Referencia	Servidor productivo con suficiente memoria

La instancia desplegada actualmente usa **Qwen2.5-7B-Instruct-Q4_K_M**. Para evitar desbordamientos silenciosos de contexto, `agent.py` cuenta tokens con el endpoint real `/v1/tokenize` antes de comprimir o truncar el historial, en lugar de estimar con una relación fija de caracteres por token.

Modelos evaluados y descartados

Se probaron varios modelos <8B buscando una opción más ligera. Ninguno alcanzó la fiabilidad de function calling de Qwen2.5-7B-Instruct.

Modelo	Tamaño	tok/s	Function calling	Veredicto
Qwen3-8B Q3_K_M	3.9 GB	14.9	Funciona	Descartado — más lento que Qwen2.5-7B Q4_K_M
Qwen3-4B (+jinja)	2.4 GB	42	Inconsistente	Descartado — responde de memoria, no usa tools
Qwen2.5- Coder-3B	1.8 GB	~33	Falla	Descartado — responde de memoria, no usa tools
Phi-4- Mini 3.8B	2.4 GB	46	Falla	Descartado — se niega a ejecutar comandos

Modelo	Tamaño	tok/s	Function calling	Veredicto
Llama-3.2-3B	1.9 GB	37	Falla	Descartado — alucina tool calls con paths falsos
Gemma-3-4B-IT	2.4 GB	16	Falla	Descartado — alucina tool calls

Conclusión: **modelos <7B no son fiables para function calling en sysadmin.** Qwen3-8B y Qwen2.5-7B-Instruct son el sweet spot para CPU.

Funcionalidades v2.2

Funcionalidad	Estado
Function calling nativo vía llama.cpp	[OK]
13 tools: run_command, read_file, write_file, web_search, git_operation, mcp_call, run_playbook, process_start, process_send, process_close, process_list, cloud_reasoning, get_context_usage	[OK]
Bucle conversacional con hasta 10 turnos	[OK]
tool→LLM	
Planificación y ejecución multi-paso sin intervención	[OK]
Memoria procedural (Skill-Pro, ICML 2026)	[OK]
Compresión real de contexto vía /v1/tokenize	[OK]
Recuperación de errores (apt/apt-get, locks)	[OK]
Sesión persistente entre reinicios	[OK]
Historial readline (flechas arriba/abajo, cursor)	[OK]
Ctrl+C interrumpe el turno actual sin salir	[OK]
Setup wizard: local/cloud/hybrid con 7 proveedores	[OK]
Selección de modelo local (Qwen3-8B / Qwen2.5-7B) con auto-download	[OK]
cloud_reasoning para modo híbrido	[OK]
Sesiones y memoria procedural separadas por modo	[OK]
Compresión por modo: 95% de 8K (local/híbrido), 50% del límite cloud	[OK]
API keys aisladas en ~/.aios/.env	[OK]
ISO integration: systemd units + firstboot + launch wrapper	[OK]

Funcionalidad	Estado
Anti-bucle (3 tool calls repetidas)	[OK]
Fallback a local si API key vacía	[OK]
Seguridad OWASP: allowlist, confirmación destructiva, validación de input, audit log	[OK]
README.md y PDF ejecutivo actualizados	[OK]

Sesiones y cache separadas por modo

Cada modo mantiene su propia sesión y memoria procedural; no se comparte contexto entre modos:

- `data/session_local.json`, `data/session_cloud.json`, `data/session_hybrid.json`
- `data/skills_memory_local.json`, `data/skills_memory_cloud.json`, `data/skills_memory_hybrid.json`

Compresión por modo

Modo	Ventana de contexto	Historial reservado	Compresión
LOCAL	32768 tokens (por defecto)	~31129 tokens (95%)	Conteo real de tokens vía <code>/v1/tokenize</code>
CLOUD	<code>context_limit</code> del proveedor	50% del <code>context_limit</code>	Misma estrategia de conteo
HÍBRIDO	32768 tokens (local)	~31129 tokens (95%)	Compresión local; <code>cloud_reasoning</code> decide cuándo delegar

Tool `cloud_reasoning`

Duodécima tool, disponible solo en modo híbrido. Envía al cloud el prompt del usuario más los últimos 10 mensajes del contexto local:

- Variables de entorno: `AIOS_CLOUD_ENDPOINT` y `AIOS_API_KEY`.
- Timeout: 120 s.
- Temperatura: 0.3.
- Casos de uso: planes multi-paso complejos, revisión de código o análisis que superan la capacidad del modelo local de 7B.

Tool `get_context_usage`

Decimotercera tool. Muestra tokens usados vs. el máximo configurado (`tokens_used`, `max_tokens`, `usage_pct`), permitiendo monitorizar el crecimiento de la sesión.

Anti-bucle

Si la misma tool con los mismos argumentos se repite 3 veces consecutivas, el agente pregunta **Abort the task?** (y/N) con timeout de 10 s. Responder y cancela la tarea; si se agota el tiempo continúa.

Correcciones recientes

- `get_context_usage`: eliminada referencia indefinida a `MAX_HISTORY_TOKENS`.
- API key vacía en modo cloud/híbrido cae a local de forma elegida.
- Detección anti-bucle (3 tool calls repetidas).
- Nota de idiomas actualizada: EN, ZH y ES probados; otros no garantizados.
- Auto-detección de RAM con escalado de contexto.
- Setup wizard completamente en inglés.
- Entrada visible para pegar API key sin problemas de portapapeles en terminal.

Arquitectura

`chat.py` -- `agent.py` -- `tools.py` -- Qwen3-8B / Qwen2.5-7B-Instruct vía llama.cpp:8083

- `setup.py`: wizard de configuración inicial.
- `chat.py`: bucle interactivo con readline y slash commands.
- `agent.py`: orquestador de function calling y memoria procedural.
- `tools.py`: definición y manejadores de herramientas.
- `memory.py`: cache procedural Skill-Pro.
- `playbook.py`: runner de playbooks YAML.
- `process.py`: gestión de procesos interactivos con PTY.
- `scripts/launch_llama.py`: launcher systemd para llama-server.
- `scripts/firstboot.sh`: setup de primer arranque para ISO.
- `systemd/aio-llama.service`: unidad del servidor de modelo.
- `systemd/aio-agent.service`: unidad de la interfaz de chat.

Uso

Configurar la primera vez:

```
python3 setup.py
```

Chat interactivo:

```
python3 chat.py
```

Ejemplos:

```
> show disk usage
> read /var/log/syslog
> write a backup script at /tmp/backup.sh
> restart nginx
```

Escribe `exit`, `quit` o `salir` para terminar. `Ctrl+C` durante un turno lo interrumpe y vuelve al prompt.

Seguridad

- Tool allowlist: rechaza `rm -rf /`, `dd` a dispositivos de bloque, `mkfs.*`, `fdisk`, `chmod 000`, etc.
- Human-in-the-loop: comandos destructivos (`rm`, `sudo rm`, `> /dev/sd*`, `format`, `dd`) piden confirmación (N por defecto, timeout 10 s).
- Validación de input: sanitiza caracteres de control, límite 1000 caracteres, detecta inyección de system prompt; los rechazos se registran en `audit.jsonl`.
- Protección de rutas: `write_file` bloquea `/etc`, `/boot`, `/sys`, `/proc`, `/dev`.
- Log de auditoría: cada invocación de tool y rechazo queda en `audit.jsonl`.
- API keys aisladas en `~/.aios/.env` y excluidas del control de versiones.
- Análisis completo en `SECURITY.md`.

Historial del proyecto

Semana 1 — RAG + híbrido

- Se empezó con **Qwen3-0.6B** + ChromaDB (e5-large embeddings) + corpus procedural de **955 documentos**.
- Batería de pruebas: solo **7/43 PASS (16%)**.
- Se intentó fine-tune del 0.6B con dataset SRE (500 ejemplos) vía Unsloth en GPU Lambda.
- El fine-tune falló por incompatibilidades de versiones (`trl`, `transformers`).
- Se abandonó el enfoque RAG por frágil y caro.

Semana 2 — Reset: function calling puro

- Se borraron los proyectos híbridos (~10 GB) y se empezó de cero.
- Se instaló **llama.cpp** compilado para CPU con `-j16`.
- Se descargó **Qwen3-8B** (bartowski instruct GGUF).
- Se construyó un agente en 3 archivos (~200 líneas) con function calling nativo.
- Tools iniciales: `run_command`, `read_file`, `write_file`.
- El agente funcionó a **17 tok/s**.

Semana 3 — Features por capas

- Memoria procedural (Skill-Pro, ICML 2026): cache JSON para respuestas repetitivas.
- Planificación multi-paso: prompt con `EJECUTA sin explicar`.
- Compresión de contexto: contar tokens reales vía `/v1/tokenize`.
- Recuperación de errores: reintentos con `apt-get` si `apt` falla y gestión de locks de `apt`.
- Sesión persistente (`data/session.json`) + historial readline (`data/.chat_history`).
- Log de auditoría (`audit.jsonl`).

Semana 3 — Tools avanzadas

- `web_search` vía Firecrawl local.
- `git_operation` (status, commit, push, diff, log).
- `mcp_call` (conectar APIs externas vía MCP).
- `run_playbook` (ejecutar YAML secuencialmente).
- `process_start / send / close / list` (procesos interactivos con PTY).

Week 3 — Seguridad (OWASP)

- Tool allowlist: `rm -rf /`, `dd`, `mkfs`, `fdisk`, `chmod 000` bloqueados.
- Human-in-the-loop: comandos destructivos piden confirmación.
- Input validation: sanitizar input, límite 1000 chars, anti-inyección.
- Se creó `SECURITY.md` con análisis OWASP completo.

Week 4 — Evaluación de modelos y setup wizard

- Se probaron varios modelos <8B; todos fallaron en function calling fiable para sysadmin.
- **Qwen2.5-7B-Instruct Q4_K_M** elegido como modelo definitivo.
- Se añadió `setup.py` con modos local/cloud/hybrid, selección de proveedor, API key y asignación automática de CPU.
- Se añadió Ctrl+C para interrumpir el turno actual sin salir del chat.

Week 5 — ISO integration y fixes de fiabilidad

- Nuevo modelo por defecto **Qwen3-8B** con descarga automática desde HuggingFace.
- API keys movidas de `config.yaml` a `~/.aios/.env` (`.gitignore`).
- Añadidos `systemd/aios-llama.service` y `systemd/aios-agent.service`.
- Añadido `scripts/launch_llama.py` para leer `~/.aios/config.yaml` y lanzar llama-server.
- Añadido `scripts/firstboot.sh` como wizard de primer arranque para ISO.
- Corregida referencia indefinida `MAX_HISTORY_TOKENS` en `get_context_usage`.
- Fallback elegido a local cuando la API key está vacía.

- Anti-bucle: 3 tool calls repetidas activan confirmación de aborto.
- Nota de idiomas: EN, ZH y ES probados.
- Wizard en inglés y entrada visible de API key.

Estado final

- Modelo: **Qwen3-8B Q4_K_M** (por defecto) o **Qwen2.5-7B-Instruct Q4_K_M** (alternativa)
- Servidor: llama.cpp en :8083, --jinja, -c 8192, -t 14
- Velocidad: **57/20 tok/s** prompt/gen
- 13 tools: `run_command`, `read_file`, `write_file`, `web_search`, `git_operation`, `mcp_call`, `run_playbook`, `process_start`, `process_send`, `process_close`, `process_list`, `cloud_reasoning`, `get_context_usage`
- Memoria: cache procedural Skill-Pro
- Modos: local / cloud / híbrido (configurados por `setup.py`)
- ISO: systemd units + firstboot + launch wrapper
- Seguridad: allowlist OWASP, confirmación destructiva, validación de input, audit log
- Repo: github.com/ccarrillomanzanares/aio-agent

Documento generado el 23 de julio de 2026.